

Project 1: Building the LMS Star Schema on BigQuery

Companion to Part 7A — Data Warehousing & ETL

This project builds the LMS analytics warehouse introduced in Section 10 of the main 7A article: a Kimball-style star schema on BigQuery, with SCD Type 2 history on *book_dim*, partitioning and clustering that take advantage of BigQuery's MPP architecture (Section 7), and validation against the exact quarter-over-quarter query used in Section 5's architecture comparison.

1. Overview & Objectives

By the end of this project, you will have:

- A working BigQuery dataset with a Kimball-style star schema: ***lending_fact***, ***reservation_fact***, and conformed ***book_dim***, ***member_dim***, ***branch_dim***, ***date_dim*** tables
- SCD Type 2 implemented and demonstrated on ***book_dim.category***, with both the pre- and post-reclassification versions independently queryable
- Partitioning and clustering configured on the fact tables, with a measured before/after query performance comparison
- The exact quarter-over-quarter lending-trend query from Section 5 of the main article, run and validated against real data

2. Prerequisites

Requirement	Notes
Google Cloud project with BigQuery API enabled	Free tier / sandbox is sufficient for this project's data volume
bq CLI or BigQuery Console access	Examples below use standard SQL via the Console or bq CLI interchangeably
Sample LMS operational data	A CSV/Parquet export of book, member, branch, and lending records — synthetic data generation script provided in the companion CLAUDE.md

Phase 1: Schema Design & DDL

Create the dataset and all five tables. Grain, per Section 10 of the main article: one row per lending event, one row per reservation event.

```
CREATE SCHEMA IF NOT EXISTS lms_warehouse
  OPTIONS (location = 'US');

-- Dimension: date_dim (standard calendar dimension, pre-populated)
CREATE TABLE lms_warehouse.date_dim (
  date_key      INT64,
  full_date     DATE,
  day_of_week   STRING,
  month         INT64,
  quarter       INT64,
  year          INT64
);

-- Dimension: branch_dim
CREATE TABLE lms_warehouse.branch_dim (
  branch_key    INT64,
  branch_id     STRING,
  branch_name   STRING,
  region        STRING
);

-- Dimension: member_dim
CREATE TABLE lms_warehouse.member_dim (
  member_key    INT64,
  member_id     STRING,
  member_name   STRING,
  home_branch_id STRING
);

-- Dimension: book_dim -- SCD Type 2 on `category` (Section 4 / Deep Dive (a))
CREATE TABLE lms_warehouse.book_dim (
  book_key      INT64,
  book_id       STRING,
  title         STRING,
  category      STRING,
  publisher_name STRING,
  effective_start_date DATE,
  effective_end_date   DATE,
  is_current      BOOL
);
```

```
-- Fact: lending_fact -- grain: one row per lending event
CREATE TABLE lms_warehouse.lending_fact (
  lending_id          INT64,
  book_key            INT64,
  member_key          INT64,
  branch_key          INT64,
  date_key            INT64,
  lending_date        DATE,
  loan_duration_days  INT64,
  fine_amount         NUMERIC
);

-- Fact: reservation_fact -- grain: one row per reservation event
CREATE TABLE lms_warehouse.reservation_fact (
  reservation_id      INT64,
  book_key            INT64,
  member_key          INT64,
  branch_key          INT64,
  date_key            INT64,
  reservation_date    DATE,
  time_to_fulfillment_hours INT64,
  is_cancelled        BOOL
);
```

Phase 2: Partitioning & Clustering

This is where BigQuery's MPP architecture (Section 7 of the main article) actually gets put to work: partitioning lets the query engine skip entire date ranges before scanning anything, and clustering sorts data within each partition so filters on other frequently-queried columns skip further still.

```
-- Re-create lending_fact partitioned by month on lending_date, clustered
-- by branch_key and book_key -- the two columns most Section 5-style
-- analytical queries filter or group by.

CREATE TABLE lms_warehouse.lending_fact_optimized
PARTITION BY DATE_TRUNC(lending_date, MONTH)
CLUSTER BY branch_key, book_key
AS SELECT * FROM lms_warehouse.lending_fact;
```

2.1 Measuring the Difference

Run the same query against both tables and compare bytes processed (visible in the BigQuery Console's query validator, or via `bq query --dry_run`):

```
-- Against the naive (unpartitioned, unclustered) table:
SELECT branch_key, COUNT(*) AS total_lendings
FROM lms_warehouse.lending_fact
WHERE lending_date BETWEEN '2026-01-01' AND '2026-03-31'
GROUP BY branch_key;
-- Dry-run: scans the entire table's date range -- every partition-month ever loaded.

-- Against the partitioned + clustered table:
SELECT branch_key, COUNT(*) AS total_lendings
FROM lms_warehouse.lending_fact_optimized
WHERE lending_date BETWEEN '2026-01-01' AND '2026-03-31'
GROUP BY branch_key;
-- Dry-run: scans only the 2026-01, 2026-02, 2026-03 partitions, and within
-- those, benefits from branch_key clustering for the GROUP BY.
```

Table Variant	Bytes Processed (dry-run)	Relative Cost
Naive (no partition/cluster)	Scans full table history	Baseline (highest)
Partitioned + clustered	Scans only 3 of N monthly partitions	Proportional to date range queried, typically a fraction of baseline

Key Insight: Record your own actual dry-run byte counts here once you've loaded a realistic data volume — the relative improvement is the point of this phase, not a specific number, since it scales with how much historical data you've loaded and how narrow your date filter is.

Phase 3: SCD Type 2 Implementation Walkthrough

Demonstrating the exact SCD Type 2 mechanics from Deep Dive (a), Section 2.3, against a real row in this project's *book_dim* table.

```
-- Starting state: one current row for book_id 'B-10293'
INSERT INTO lms_warehouse.book_dim
  (book_key, book_id, title, category, publisher_name,
   effective_start_date, effective_end_date, is_current)
VALUES
  (1001, 'B-10293', 'The Giver', 'Fiction', 'HMH Books',
   '2024-01-01', NULL, TRUE);

-- Reclassification event: 'Fiction' -> 'Young Adult'

-- Step 1: expire the current row
UPDATE lms_warehouse.book_dim
SET effective_end_date = CURRENT_DATE() - 1,
    is_current = FALSE
WHERE book_id = 'B-10293' AND is_current = TRUE;

-- Step 2: insert the new version
INSERT INTO lms_warehouse.book_dim
  (book_key, book_id, title, category, publisher_name,
   effective_start_date, effective_end_date, is_current)
VALUES
  (1002, 'B-10293', 'The Giver', 'Young Adult', 'HMH Books',
   CURRENT_DATE(), NULL, TRUE);
```

3.1 Verifying Both Versions Are Independently Queryable

```
-- A lending event that happened BEFORE the reclassification should still show 'Fiction':
SELECT f.lending_id, f.lending_date, b.category
FROM lms_warehouse.lending_fact f
JOIN lms_warehouse.book_dim b
  ON f.book_key = b.book_key
WHERE f.book_key = 1001;  -- the pre-reclassification surrogate key
-- Expected: category = 'Fiction'

-- A lending event happening AFTER should show 'Young Adult':
SELECT f.lending_id, f.lending_date, b.category
FROM lms_warehouse.lending_fact f
JOIN lms_warehouse.book_dim b
  ON f.book_key = b.book_key
WHERE f.book_key = 1002;  -- the post-reclassification surrogate key
-- Expected: category = 'Young Adult'
```

Key Insight: If both queries return the correct, different category values for the same underlying book, SCD Type 2 is working as intended: history is preserved, and each fact row is joined against whichever dimension version was live when that lending event actually happened.

Phase 4: Validation Queries

Running the exact query from Section 5's Kimball comparison row, against this project's finished schema.

```
SELECT
  b.category,
  br.branch_name,
  d.quarter,
  d.year,
  COUNT(*) AS total_lendings
FROM lms_warehouse.lending_fact_optimized f
JOIN lms_warehouse.book_dim b    ON f.book_key = b.book_key
JOIN lms_warehouse.branch_dim br ON f.branch_key = br.branch_key
JOIN lms_warehouse.date_dim d    ON f.date_key = d.date_key
GROUP BY b.category, br.branch_name, d.quarter, d.year
ORDER BY d.year, d.quarter, total_lendings DESC;
```

This is the same query shape used throughout Section 5 of the main article to compare Inmon, Kimball, and Data Vault — running it here confirms the star schema built in this project answers it directly, in a single pass, with one join per dimension.

4. Verification Checklist

- All five tables created successfully with no schema errors
- **`lending_fact_optimized`** is partitioned by month and clustered by `branch_key`, `book_key`
- Dry-run byte comparison between the naive and optimized tables shows a measurable reduction when filtering by date range
- SCD Type 2 update executed on *book_dim*; both pre- and post-reclassification category values are independently queryable against the correct lending events
- The Section 5 quarter-over-quarter validation query runs successfully and returns results grouped by category, branch, and quarter

Appendix: Full DDL Reference

All table definitions from this project in one place, for copy-paste use independent of the phase-by-phase walkthrough above.

```
CREATE SCHEMA IF NOT EXISTS lms_warehouse OPTIONS (location = 'US');

CREATE TABLE lms_warehouse.date_dim (
  date_key INT64, full_date DATE, day_of_week STRING,
  month INT64, quarter INT64, year INT64
);

CREATE TABLE lms_warehouse.branch_dim (
  branch_key INT64, branch_id STRING, branch_name STRING, region STRING
);

CREATE TABLE lms_warehouse.member_dim (
  member_key INT64, member_id STRING, member_name STRING, home_branch_id STRING
);

CREATE TABLE lms_warehouse.book_dim (
  book_key INT64, book_id STRING, title STRING, category STRING,
  publisher_name STRING, effective_start_date DATE,
  effective_end_date DATE, is_current BOOL
);

CREATE TABLE lms_warehouse.lending_fact (
  lending_id INT64, book_key INT64, member_key INT64, branch_key INT64,
  date_key INT64, lending_date DATE, loan_duration_days INT64, fine_amount NUMERIC
);

CREATE TABLE lms_warehouse.reservation_fact (
  reservation_id INT64, book_key INT64, member_key INT64, branch_key INT64,
  date_key INT64, reservation_date DATE,
  time_to_fulfillment_hours INT64, is_cancelled BOOL
);

CREATE TABLE lms_warehouse.lending_fact_optimized
PARTITION BY DATE_TRUNC(lending_date, MONTH)
CLUSTER BY branch_key, book_key
AS SELECT * FROM lms_warehouse.lending_fact;
```

This project is a companion to Part 7A of “Developing Intuition on Building Blocks — for Systems Design.” See the accompanying [CLAUDE.md](#) for a phased Claude Code implementation plan, and [Project 2](#) for the ETL/ELT pipeline that populates this warehouse from the LMS’s operational database.